[1] Multi-processor machines can, nevertheless, apply optimizations where some operations are executed in parallel across different cores. This level of parallelization, however, is limited by the number of cores on a chip – currently at most in the order of a hundred, for the most powerful servers.

[2] References: https://people.eecs.berkeley.edu/~rcs/research/interactive_latency.html and originally http://norvig.com/21-days.html#answers.

[3] Considering a WAN or high-performance cloud service. Local clusters in datacenters, when properly configured, can lower this latency by two orders of magnitude, down to 1ms.

[4] It can both reduce the number of iterations needed, and reduce the number of operations performed in each iteration.

[5] The Cartesian Product between two sets is the multiplication of the two sets to form a new set, containing all the ordered pairs such that the first element belongs to first set and second element belongs to the second set.

[6] Tipically a per-key count on mappers' output would be done by a third abstraction, the combiners, that are sort of mini-reducers operating on the same machines as mappers, and only on the output of individual mappers: so that instead of a list with a ton of entry with value 1, the mapper would send out a list with just a few entries, reducing both bandwidth and the workload for reducers.

[7] Methods of an immutable data structures, rather than changing the object `A` on which they are called, create a new object `B` whose state is the result of applying the called method to `A`. For instance, the method that appends an element to a list `L1` would create a brand-new list `L2` with `|L1| + 1` elements, and leave `L1` unchanged.

[8] A pure function is any function that doesn't have a side effect: takes 0, 1 or more inputs, and returns an output (possibly in the form of a tuple), without relying on any change to the input or to the global state. In a sense, a pure function is exactly like a mathematical function.

[9] That's also the reason why, as discussed in the previous sub-sections, it can't be applied to all those problems that cannot be stated in a way that eliminates shared state.

[10] Notice that here we will list k-means steps from the easiest to the hardest to parallelize.

[11] Keep in mind that this is not a real implementation, and so we can take some shortcuts to try and explain the fundamental ideas more clearly; a Hadoop MapReduce job, for instance, would look different.

[12] A good starting point to delve into Iterative Map Reduce can be https://cloudmesh.github.io/classes/icloud/2017/course/iterative-mapreduce.html

[13] Technically, computing the mean of a set of values could also be distributed: a combiner would need to take as input the shards' centroids and number of points in each shard, to compute the overall center of mass.

[14] Gradient descent is an optimization algorithm for finding (local) minimums of a function `F`: it explores the function's domain in a systematic way, taking steps proportional to the gradient of `F` at current point `X`, which (simplifying) can be geometrically interpreted as the direction of greatest change for `F` at `X`.